PasswordStore Audit Report

Tim

April 22, 2026

Contents

PasswordStore Audit Report **2**

Table of Contents **2**

Protocol Summary **2**

Disclaimer **3**

Risk Classification **3**

Audit Details **3**

Scope 3

Roles 3

Executive Summary **3**

Issues found 4

Findings **4**

High **4**

[H-1] Storing the password onchain makes it visible to anyone 4

[H-2] `PasswordStore::setPassword` has no access control meaning a non-owner can change the password 5

Informational **5**

[I-1] `PasswordStore::getPassword` natspec indicates a parameter that does not exist 5

PasswordStore Audit Report

Prepared by: Tim
Lead Auditor: Tim
Assisting Auditors: None
Date: April 22, 2026
Version: 1.0

Table of Contents

- PasswordStore Audit Report
- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
  - [H-1] Storing the password onchain makes it visible to anyone
  - [H-2] `PasswordStore::setPassword` has no access control meaning a non-owner can change the password
- Informational
  - [I-1] `PasswordStore::getPassword` natspec indicates a parameter that does not exist

Protocol Summary

Protocol does X, Y, Z

Disclaimer

The team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

Roles

- **Owner:** The user who can set the password and read the password.
- **Outsiders:** No one else should be able to set or read the password.

Executive Summary

No analytic package was used for this audit.

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	0
Informational	1

Findings

High

[H-1] Storing the password onchain makes it visible to anyone

Description: All data stored onchain is visible to anyone and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be a private variable and only accessed through the `PasswordStore::getPassword` function, which is intended to be called only by the owner of the contract.

One such method of reading this data is shown below.

Impact: Anyone can read the password, severely breaking the intended functionality of the protocol.

Proof of Concept: The test below demonstrates that anyone can read the password directly from the blockchain. After the contract was deployed on a local chain, the storage slot for the password was queried and its `bytes32` value parsed to a string, revealing the password exactly as the owner wrote it.

Recommended Mitigation: The overall architecture of the contract should be reconsidered. One approach is to encrypt the password off-chain and store only the encrypted value on-chain. This would require the user to remember a separate off-chain key to decrypt it. Additionally, the view function should likely be removed to prevent the user from accidentally exposing the decryption password in a transaction.

[H-2] `PasswordStore::setPassword` has no access control meaning a non-owner can change the password

Description: The `PasswordStore::setPassword` function is an external function with no access control. The natspec and overall purpose of the contract state that this function should allow only the owner to set a new password.

```
function setPassword(string memory newPassword) external {
    // @Audit - No access control here
    s_password = newPassword;
    emit SetNetPassword();
}
```

Impact: Anyone can set or change the password, severely breaking the intended functionality of the contract.

Proof of Concept: Add the following test to `passwordStore.t.sol`:

```
function test_anyone_can_set_password(address randomAddress) public {
    vm.assume(randomAddress != owner);

    vm.prank(randomAddress);
    string memory expectedPassword = "tim";
    passwordStore.setPassword(expectedPassword);

    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEq(actualPassword, expectedPassword);
}
```

Recommended Mitigation: Add an access control check to the `setPassword` function:

```
if (msg.sender != owner) {
    revert PasswordStore_NotOwner();
}
```

Informational

[I-1] `PasswordStore::getPassword` natspec indicates a parameter that does not exist

Description: The natspec for `getPassword` references a `newPassword` parameter that does not exist in the function signature. The function signature is `getPassword()` but the natspec implies `getPassword(string)`.

```
/*
 * @notice This function allows only the owner to set a new password.
 * @param newPassword The new password to set.
 */
function setPassword(string memory newPassword) external {
```

Impact: The natspec is incorrect, reducing code clarity and documentation quality.

Recommended Mitigation: Remove the incorrect natspec line:

```
-  * @param newPassword The new password to set.
```